МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАХІДНОУКРАЇНСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
Факультет комп’ютерних інформаційних технологій
Кафедра комп’ютерних наук

ЗВІТ

про виконання лабораторної роботи №3
з дисципліни «Методи аналізу інтервальних даних»
на тему: «Програмна система для пошуку мінімуму функції методом Ньютона»
Варіант 1

Виконав:
студент групи ІПЗМПм-11
Боднар А.

Перевірила:
к.т.н., доцент
Крепич С. Я.

Тернопіль – 2026

МЕТА РОБОТИ

Вивчити інтервальний метод Ньютона для гарантованої локалізації нуля функції однієї дійсної змінної; навчитися будувати інтервальне розширення функції та її похідної; розробити програмну систему, яка за введеними функцією, похідною і початковим проміжком знаходить інтервал заданої ширини, що містить шукану точку.

ЗАВДАННЯ

Розробити програмну систему для знаходження мінімуму функції на заданому проміжку з використанням інтервального методу Ньютона. У програмній системі передбачити:

- зручний графічний інтерфейс;
- перевірку коректності введених даних;
- введення функції та її похідної;
- введення початкового проміжку;
- виведення результуючого проміжку;
- виведення кількості ітерацій і часу їх виконання;
- виведення графіка функції на початковому проміжку з позначенням результуючого інтервалу.

ВИХІДНІ ДАНІ

Таблиця 1 – Дані варіанта 1

Параметр	Значення
Функція	f(x) = sin(2·x^2) − cos(x)·x^3
Початковий проміжок	X_0 = [−1,4; −1,05]
Точність (ширина інтервалу)	ε = 0,0001

ТЕОРЕТИЧНІ ВІДОМОСТІ

Методи локалізації нулів функції дозволяють знайти інтервал якомога меншої ширини, який гарантовано містить нуль функції f(x) з початково заданого

проміжку $X_0 = [x_0^-; x_0^+]$. Початкова умова — функція на кінцях проміжку має різні знаки: $f(x_0^-) < 0 < f(x_0^+)$ або $f(x_0^-) > 0 > f(x_0^+)$.

Інтервальний метод Ньютона будує послідовність вкладених інтервалів

$$X_{k+1} = \{ m(X_k) - f(m(X_k)) / M \} \cap X_k, \quad k \geq 0,$$

де $m(X_k) = (x_k^- + x_k^+)/2$ — середина інтервалу, $M = [m_1; m_2]$ — інтервал, що містить усі значення похідної $f'(x)$ на X_k. Якщо $0 \notin M$, то кожен нуль функції з X_k належить і X_{k+1}, а ширина інтервалу на кожному кроці зменшується щонайменше вдвічі:

$$d(X_{k+1}) \leq ½ \cdot (1 - m_1/m_2) \cdot d(X_k).$$

Інтервал M у програмі обчислюється як природне інтервальне розширення похідної: у формулі $f'(x)$ змінна замінюється інтервалом X_k, а кожна арифметична дія та елементарна функція (sin, cos, exp, ln, степінь, корінь) — її інтервальним аналогом, що дає множину всіх можливих значень. Наприклад, для sin враховується, чи потрапляють у інтервал точки максимуму $\pi/2 + 2\pi k$ і мінімуму $-\pi/2 + 2\pi k$. Так M гарантовано містить справжній діапазон f' на X_k, тому нуль не може бути втрачений. Якщо ж похідна на X_k змінює знак ($0 \in M$), крок Ньютона не визначений, і інтервал ділиться навпіл — залишається половина, на кінцях якої f має різні знаки.

Похибка оцінювання наближеного значення нуля $m(X_k)$ визначається за формулою

$$\varepsilon = |X_k^+ / m(X_k) - 1| \cdot 100\ \%.$$

ХІД РОБОТИ

1. Похідна функції та перевірка початкової умови

$$f'(x) = 4 \cdot x \cdot \cos(2 \cdot x\text{^}2) + \sin(x) \cdot x\text{^}3 - 3 \cdot x\text{^}2 \cdot \cos(x).$$

На кінцях проміжку $f(-1{,}4) = -0{,}2358$ і $f(-1{,}05) = 1{,}3815$ мають різні знаки, отже, на X_0 є точка, де $f(x) = 0$. Так само для всіх 25 варіантів завдання функція змінює знак на заданому проміжку, тому шуканий інтервал локалізується методом Ньютона з лекції 5 за введеними $f(x)$ і $f'(x)$. Правильність введеної похідної програма перевіряє порівнянням з чисельною похідною $f(x)$ у 21 точці проміжку.

2. Перша ітерація вручну

$$m(X_0) = (-1{,}4 + (-1{,}05))/2 = -1{,}225; \quad f(m(X_0)) = 0{,}762955;$$

Інтервальне розширення похідної на X_0 (обчислене за правилами інтервальної арифметики для кожної дії у формулі f′):

$$M = f'(X_0) = [0{,}567085;\ 7{,}741908], \quad 0 \notin M;$$

$$N = m - f(m)/M = -1{,}225 - (0{,}762955)/[0{,}567085;\ 7{,}741908] =$$

$$[-2{,}570398;\ -1{,}323548];$$

$$X_1 = N \cap X_0 = [-2{,}570398;\ -1{,}323548] \cap [-1{,}4;\ -1{,}05] = [-1{,}4;\ -1{,}323548].$$

Перевірка умови з лекції:

$d(X_1) = 0{,}076451 \le ½\cdot(1 - m_1/m_2)\cdot d(X_0) = 0{,}162181$ — умова виконується.

3. Результати ітерацій

Повний хід обчислень наведено в таблиці 2.

Таблиця 2 – Ітерації інтервального методу Ньютона

k	**$m(X_{k-1})$**	**f(m)**	**$M = f'(X_{k-1})$**	**X_k**
1	−1,225	7,6296e−1	[0,567085; 7,741908]	[−1,4; −1,323548]
2	−1,36177436	−1,3317e−2	[4,57865; 7,047965]	[−1,359885; −1,358865]
3	−1,35937529	8,6867e−4	[5,899969; 5,932621]	[−1,359523; −1,359521]

Після 3 ітерацій ширина інтервалу $d(X^*) = 8{,}103e{-}7 < \varepsilon$, тому обчислення зупинено. Результуючий інтервал $X^* = [-1{,}359523;\ -1{,}359521]$, наближене значення нуля $m(X^*) = -1{,}35952211$, похибка $\varepsilon = 2{,}98e{-}5$ %.

4. Програмна реалізація

Програмну систему реалізовано як вебзастосунок мовою TypeScript з бібліотеками React і React-Bootstrap; вона працює в браузері й розміщена на GitHub Pages. Обчислювальна частина не залежить від інтерфейсу:

– interval.ts — клас Interval з інтервальними арифметичними операціями (використовувався й у попередніх роботах);

– func.ts — розбір формули f(x), введеної текстом, у дерево виразу; обчислення значення функції в точці та її природного інтервального

розширення на інтервалі (sin, cos, tan, ctg, exp, ln, lg, sqrt, abs, root, степені); перевірка введеної похідної;

– lab3.ts — дані 25 варіантів, перевірка вхідних даних (межі, точність, різні знаки f на кінцях, область визначення) і функція solveLab3, що виконує ітерації та вимірює їх час.

Інтерфейс складається з трьох кроків: введення функції, похідної, проміжку й точності; таблиця ітерацій; результат і графік. Помилки у формулах показуються одразу з позицією символу, а кнопка обчислення недоступна, доки дані некоректні. Правильність обчислень перевірено автоматичними тестами (Vitest): на прикладах з лекції 5, на всіх 25 варіантах і на перевірці, що інтервальне розширення містить усі значення функції.

Сторінка роботи на сайті: https://4ndruxa.github.io/university-labs/#/maid/lab3.

Репозиторій з кодом: https://github.com/4ndruxa/university-labs.

5. Результати роботи програми

На рисунку 1 наведено вікно введення функції, похідної та проміжку.

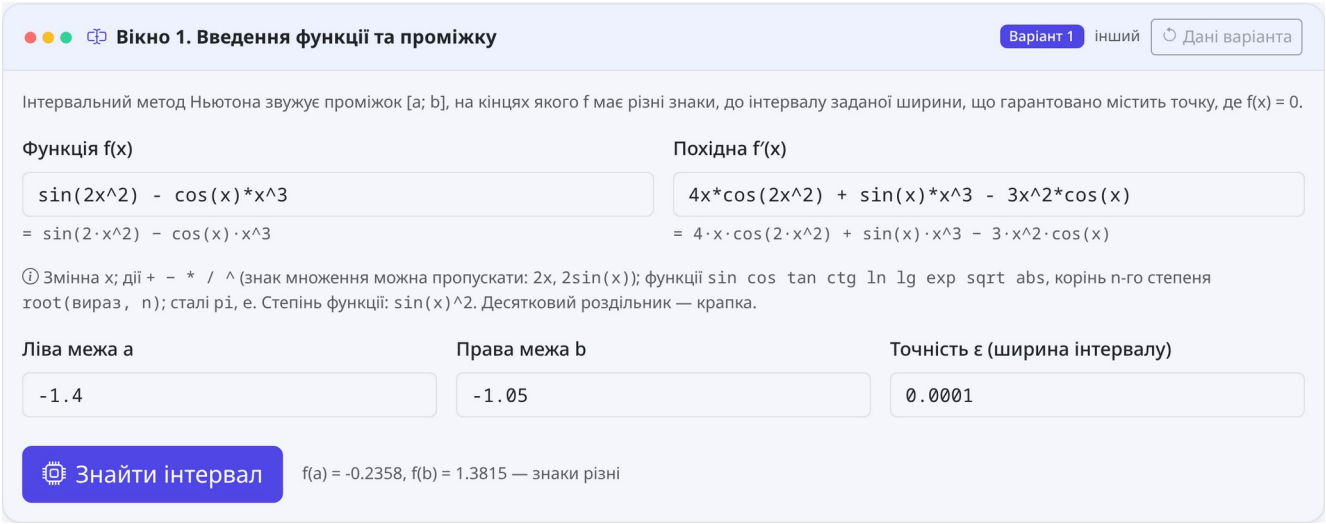


Рисунок 1 – Вікно введення вхідних даних

На рисунку 2 наведено таблицю ітерацій методу.

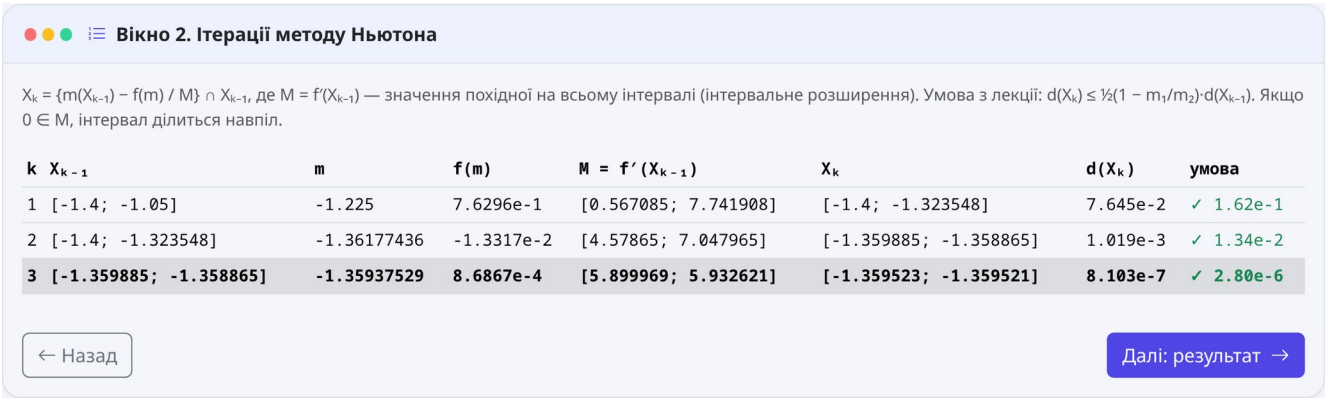


Рисунок 2 – Вікно з ітераціями методу Ньютона

На рисунку 3 наведено результуючий інтервал, кількість ітерацій, час їх виконання і графік функції на початковому проміжку з позначеним результуючим інтервалом.

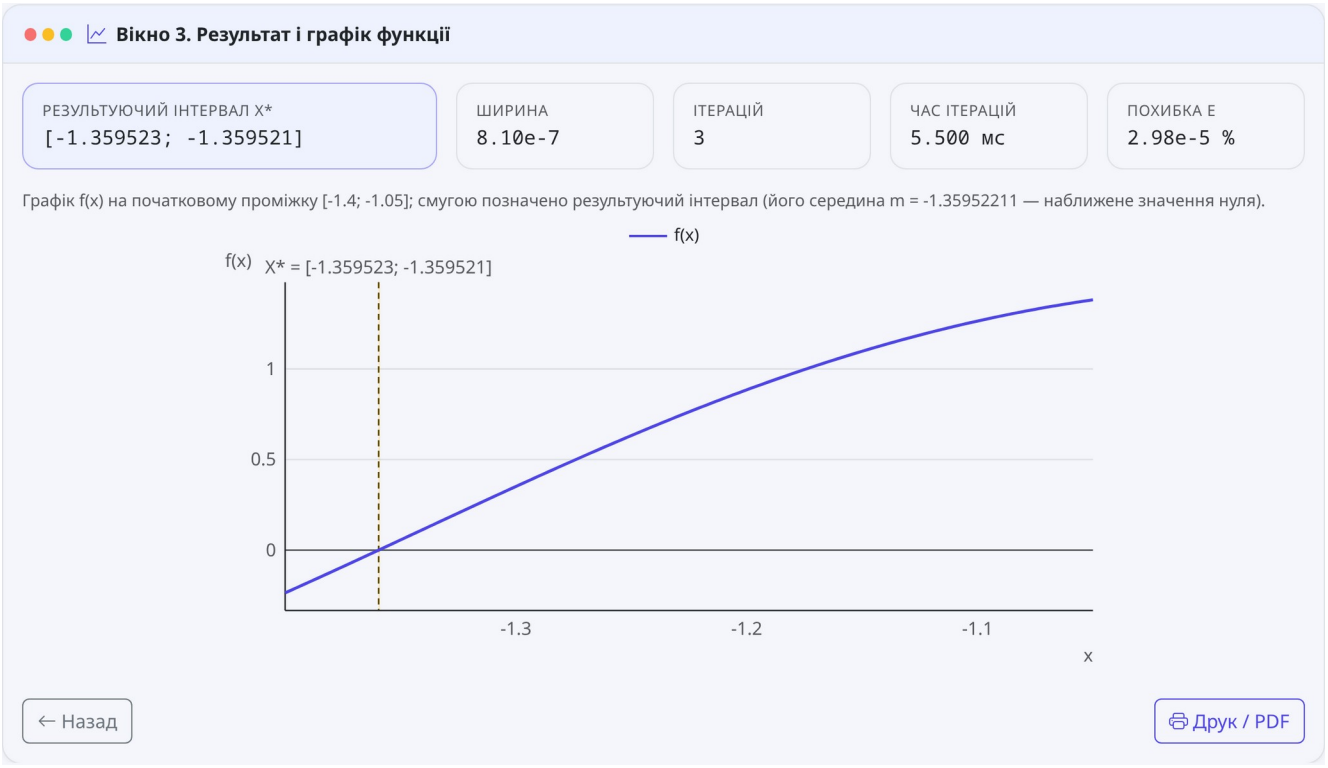


Рисунок 3 – Вікно з результатом і графіком функції

ВИСНОВКИ

У ході лабораторної роботи вивчено інтервальний метод Ньютона і розроблено програмну систему, яка за введеними функцією, її похідною та початковим проміжком гарантовано локалізує нуль функції.

Для варіанта 1 початковий проміжок шириною 0,35 за 3 ітерації звужено до інтервалу $X^* = [-1{,}359523; -1{,}359521]$ шириною $8{,}10e{-}7$. Метод збігається дуже швидко: ширина інтервалу на кожній ітерації зменшувалась у десятки й сотні разів, а умова $d(X_{k+1}) \le \tfrac{1}{2}(1 - m_1/m_2)d(X_k)$ виконувалась на всіх кроках.

Використання природного інтервального розширення похідної гарантує, що нуль функції не буде втрачено навіть тоді, коли похідна немонотонна на проміжку; ціною є дещо ширший інтервал M на першій ітерації, який швидко звужується разом з X_k.

ДОДАТОК А. ЛІСТИНГ ПРОГРАМНОГО КОДУ

Нижче наведено обчислювальну частину програми з коментарями. Повний код застосунку, разом з інтерфейсом і тестами, розміщено в репозиторії https://github.com/4ndruxa/university-labs.

Файл lab3.ts

```
// ЛР3: інтервальний метод Ньютона — локалізація нуля функції на проміжку (лекція
    5)
//   X₀ = [a; b],  f(a)·f(b) < 0
//   Xₖ₊₁ = { m(Xₖ) − f(m(Xₖ)) / M } ∩ Xₖ,   M = f'(Xₖ) — інтервальне розширення
    похідної
// Якщо 0 ∈ M (похідна на Xₖ змінює знак), крок Ньютона не визначений — ділимо Xₖ
    навпіл
// і залишаємо половину, на кінцях якої f має різні знаки.
import { DomainError, evalAt, evalInterval, parseFunction, type Node } from
    './func'
import { Interval } from './interval'

export interface Lab3Variant {
  f: string
  df: string
  a: number
  b: number
}

// Варіанти з методичних вказівок; похідні знайдено аналітично (перевіряються
    тестом)
export const LAB3_VARIANTS: readonly Lab3Variant[] = [
  { f: 'sin(2x^2) - cos(x)*x^3', df: '4x*cos(2x^2) + sin(x)*x^3 - 3x^2*cos(x)', a:
    -1.4, b: -1.05 },
  { f: 'cos(x^2)^2 + 2sin(x)', df: '-4x*cos(x^2)*sin(x^2) + 2cos(x)', a: -1, b:
    0.5 },
  { f: '1/sqrt(x) + 2sin(2x)*x', df: '-1/(2x*sqrt(x)) + 4x*cos(2x) + 2sin(2x)', a:
    2.6, b: 3.5 },
  { f: '4^cos(x^2) - 1', df: '-2x*ln(4)*sin(x^2)*4^cos(x^2)', a: -1.5, b: -0.8 },
  { f: '-0.3 + ln(x)/x', df: '(1 - ln(x))/x^2', a: 1.2, b: 1.9 },
  { f: '2cos(4x^2)^2 - 0.2', df: '-32x*cos(4x^2)*sin(4x^2)', a: 1.1, b: 1.2 },
  { f: '2^(sin(x)*cos(x)) - 1.2', df: 'ln(2)*cos(2x)*2^(sin(x)*cos(x))', a: 3.15,
    b: 3.8 },
  { f: 'x^3/ln(x) - 10', df: '(3x^2*ln(x) - x^2)/ln(x)^2', a: 1.5, b: 2.3 },
  { f: 'cos(2x)^3/sqrt(x) + 0.2', df: '-6cos(2x)^2*sin(2x)/sqrt(x) -
    cos(2x)^3/(2x*sqrt(x))', a: 1.9, b: 2.1 },
  { f: 'sin(2x^3)/root(x, 5) - 0.5', df: '6x^2*cos(2x^3)/root(x, 5) -
    sin(2x^3)/(5x*root(x, 5))', a: 0.25, b: 1 },
  { f: '3^(sin(x^2)^2) - 1.5', df: '2x*ln(3)*sin(2x^2)*3^(sin(x^2)^2)', a: 0.6, b:
    1.1 },
  { f: 'x*ln(x)/sqrt(x) + cos(x) - 0.7', df: '(ln(x) + 2)/(2sqrt(x)) - sin(x)', a:
    2.1, b: 3.1 },
  { f: 'root(cos(x)*sin(x), 7) + 0.8', df: 'cos(2x)/(7*root(cos(x)*sin(x), 7)^6)',
    a: 2.8, b: 3.05 },
  { f: '(cos(x)/sin(x))^2 - 8', df: '-2cos(x)/sin(x)^3', a: 2.4, b: 2.95 },
  { f: 'root(4^(x^2), 5) - 2', df: '2x*ln(4)/5*root(4^(x^2), 5)', a: 1.5, b:
    1.7 },
  { f: '2^sin(x)*3^x - 8', df: '2^sin(x)*3^x*(ln(2)*cos(x) + ln(3))', a: 1.1, b:
    1.9 },
```

```
  { f: 'sin(exp(5x^3))^3 - 0.7', df:
    '45x^2*exp(5x^3)*sin(exp(5x^3))^2*cos(exp(5x^3))', a: 0.16, b: 0.4 },
  { f: 'cos(ln(x)*x/6)', df: '-sin(ln(x)*x/6)*(ln(x) + 1)/6', a: 11, b: 11.75 },
  { f: 'cos(x/3)^2*root(x^3, 5) - 2', df: '-2/3*cos(x/3)*sin(x/3)*root(x^3, 5) +
    3cos(x/3)^2/(5root(x^2, 5))', a: 7.2, b: 9 },
  { f: 'sin(x)^3*root((x - 2)^3, 5) + 0.8', df: '3sin(x)^2*cos(x)*root((x - 2)^3,
    5) + 3sin(x)^3/(5root((x - 2)^2, 5))', a: 5, b: 7.2 },
  { f: '3^root(x^2 - 2, 6) - 8/x', df: 'ln(3)*x*3^root(x^2 - 2, 6)/(3root((x^2 -
    2)^5, 6)) + 8/x^2', a: 1.6, b: 3.6 },
  { f: '2sin(2x)^2 - 8x^3 - 1/4', df: '4sin(4x) - 24x^2', a: 0.13, b: 0.25 },
  { f: '2sin(x)^2 - 4x^3*cos(2x) - 0.5', df: '2sin(2x) - 12x^2*cos(2x) +
    8x^3*sin(2x)', a: 0.3, b: 0.874 },
  { f: '(6^(4x^3) + 3)^2 - 8x^2', df: '24x^2*ln(6)*(6^(4x^3) + 3)*6^(4x^3) - 16x',
    a: -2, b: -0.3 },
  { f: 'ln(x)*root((cos(x)/sin(x))^2, 5) - 3', df: 'root((cos(x)/sin(x))^2, 5)/x -
    2ln(x)*root((cos(x)/sin(x))^2, 5)/(5cos(x)*sin(x))', a: 5, b: 6.2 },
]

export const LAB3_EPS = 1e-4
export const LAB3_MAX_ITER = 100

export interface Lab3Row {
  k: number // номер ітерації (1, 2, …)
  X: Interval // X_{k-1} — інтервал на початку ітерації
  m: number // m(X_{k-1})
  fm: number // f(m)
  M: Interval | null // f'(X_{k-1}); null — похідна не обчислюється на всьому X_{k-1}
  /** 'newton' — крок Ньютона, 'bisect' — 0 ∈ M, поділ навпіл */
  kind: 'newton' | 'bisect'
  N: Interval | null // m − f(m)/M (для кроку Ньютона)
  next: Interval // X_k
  /** Умова з лекції: d(X_k) ≤ ½(1 − m₁/m₂)·d(X_{k-1}) */
  bound: number | null
  boundOk: boolean | null
}

export interface Lab3Result {
  rows: Lab3Row[]
  X: Interval // результуючий інтервал
  iterations: number
  timeMs: number
  stop: 'eps' | 'stagnation' | 'maxIter'
  /** Похибка за лекцією: |X⁺/m(X) − 1|·100 % */
  errorPct: number
}

export interface Lab3Input {
  f: Node
  df: Node
  a: number
  b: number
  eps: number
}

/** Перевірка вхідних даних; повертає список помилок (порожній — можна рахувати)
    */
export function validateLab3(inp: Lab3Input): string[] {
  const { f, df, a, b, eps } = inp
  const errors: string[] = []
  if (!Number.isFinite(a) || !Number.isFinite(b)) errors.push('Межі проміжку —
    числа')
  else if (a >= b) errors.push('Ліва межа має бути меншою за праву')
```

```
  if (!(eps > 0)) errors.push('Точність ε — додатне число')
  if (errors.length) return errors
  let fa: number, fb: number
  try {
    fa = evalAt(f, a)
    fb = evalAt(f, b)
  } catch (e) {
    return [`f(x) не обчислюється на кінцях проміжку: ${(e as Error).message}`]
  }
  if (fa * fb > 0) errors.push(`f(a) і f(b) мають однаковий знак (f(a) = $
    {fa.toPrecision(4)}, f(b) = ${fb.toPrecision(4)}) — нуль на проміжку не
    гарантований`)
  try {
    evalInterval(f, new Interval(a, b))
  } catch (e) {
    errors.push(`f(x) не визначена на всьому проміжку: ${(e as Error).message}`)
  }
  try {
    evalInterval(df, new Interval(a, b))
  } catch (e) {
    if (!(e instanceof DomainError && /містить 0/.test(e.message)))
    errors.push(`f'(x) не визначена на всьому проміжку: ${(e as Error).message}`)
  }
  return errors
}

/** f(m) як число для таблиці та знаку; f(m) як інтервал — для гарантованого кроку
    */
const pointInterval = (f: Node, m: number) => evalInterval(f, new Interval(m))

/** Інтервальний метод Ньютона. now — годинник (для тестів), за замовчуванням
    performance.now */
export function solveLab3(inp: Lab3Input, maxIter = LAB3_MAX_ITER, now: () =>
    number = () => performance.now()): Lab3Result {
  const errors = validateLab3(inp)
  if (errors.length) throw new Error(errors.join('\n'))
  const { f, df, eps } = inp
  const rows: Lab3Row[] = []
  let X = new Interval(inp.a, inp.b)
  let stop: Lab3Result['stop'] = 'maxIter'
  const t0 = now()

  for (let k = 1; k <= maxIter; k++) {
    const m = X.mid
    const fm = evalAt(f, m)
    let M: Interval | null = null
    try {
      M = evalInterval(df, X)
    } catch (e) {
      if (!(e instanceof DomainError)) throw e
    }
    let next: Interval
    let N: Interval | null = null
    let bound: number | null = null
    let kind: Lab3Row['kind'] = 'newton'
    if (M === null || M.contains(0)) {
      kind = 'bisect'
      const fl = evalAt(f, X.lo)
      next = fl * fm <= 0 ? new Interval(X.lo, m) : new Interval(m, X.hi)
    } else {
      const Fm = pointInterval(f, m)
      const q = Fm.quotients(M)
```

```
      N = new Interval(m - Math.max(...q), m - Math.min(...q))
      const lo = Math.max(N.lo, X.lo)
      const hi = Math.min(N.hi, X.hi)
      if (lo > hi) throw new Error(`На інтервалі ${X} нуля немає: N ∩ X = ∅`)
      next = new Interval(lo, hi)
      const m1 = Math.min(Math.abs(M.lo), Math.abs(M.hi))
      const m2 = Math.max(Math.abs(M.lo), Math.abs(M.hi))
      bound = 0.5 * (1 - m1 / m2) * X.width
    }
    rows.push({ k, X, m, fm, M, kind, N, next, bound, boundOk: bound === null ?
    null : next.width <= bound + 1e-15 })
    const stagnated = next.lo === X.lo && next.hi === X.hi
    X = next
    if (X.width < eps) { stop = 'eps'; break }
    if (stagnated) { stop = 'stagnation'; break }
  }

  const timeMs = now() - t0
  const mid = X.mid
  return {
    rows,
    X,
    iterations: rows.length,
    timeMs,
    stop,
    errorPct: mid === 0 ? 0 : Math.abs(X.hi / mid - 1) * 100,
  }
}

/** Розбір і розв'язання з текстових формул (сторінка й звіт) */
export function lab3FromText(fSrc: string, dfSrc: string, a: number, b: number,
    eps = LAB3_EPS) {
  return { f: parseFunction(fSrc), df: parseFunction(dfSrc), a, b, eps }
}
```

Файл func.ts

```
// Функції однієї змінної: розбір формули f(x) і обчислення — у точці (число) та
    на інтервалі
// (природне інтервальне розширення: кожна операція замінюється інтервальною).
//
//   expr    := term (('+' | '−') term)*
//   term    := unary (('·' | '/' | неявне множення) unary)*
//   unary   := ('−' | '+') unary | power
//   power   := primary ('^' unary)?                — правоасоціативний: 2^3^2 =
    2^(3^2)
//   primary := число | x | pi | e | функція | '(' expr ')'
//   функція := ім'я ('^' число)? '(' expr (',' expr)? ')'   — sin^2(x) = sin(x)^2
import { Interval } from './interval'

export type Node =
  | { k: 'num'; v: number }
  | { k: 'x' }
  | { k: 'neg'; a: Node }
  | { k: 'bin'; op: '+' | '-' | '*' | '/'; a: Node; b: Node }
  | { k: 'pow'; a: Node; b: Node }
  | { k: 'fn'; name: FnName; args: Node[] }

const FUNCTIONS = ['sin', 'cos', 'tan', 'ctg', 'ln', 'lg', 'exp', 'sqrt', 'abs',
    'root'] as const
export type FnName = (typeof FUNCTIONS)[number]
const ALIASES: Record<string, FnName> = { tg: 'tan', cot: 'ctg', log: 'ln' }
```

```
/** pos — індекс символу (з 0) */
export class FuncError extends Error {
  readonly pos: number
  constructor(message: string, pos: number) {
    super(message)
    this.name = 'FuncError'
    this.pos = pos
  }
}

/** Значення не існує: ln від недодатного, ділення на 0 тощо */
export class DomainError extends Error {
  constructor(message: string) {
    super(message)
    this.name = 'DomainError'
  }
}

// ---------- Розбір ----------

type Token =
  | { t: 'num'; v: number; pos: number }
  | { t: 'id'; v: string; pos: number }
  | { t: 'sym'; v: string; pos: number }
  | { t: 'end'; pos: number }

const NORMALIZE: Record<string, string> = { '−': '-', '–': '-', '·': '*', '×':
    '*', '÷': '/', 'π': 'pi', '²': '^2', '³': '^3', ';': ',' }

function tokenize(src: string): Token[] {
  const tokens: Token[] = []
  let i = 0
  while (i < src.length) {
    const c = src[i]
    if (/\s/.test(c)) { i++; continue }
    if (/[0-9.]/.test(c)) {
      const m = /^(\d+\.?\d*|\.\d+)([eE][+-]?\d+)?/.exec(src.slice(i))
      if (!m) throw new FuncError(`Некоректне число на позиції ${i + 1}`, i)
      tokens.push({ t: 'num', v: Number(m[0]), pos: i })
      i += m[0].length
      continue
    }
    if (/[a-zA-Za-яA-Я]/.test(c)) {
      const m = /^[a-zA-Za-яA-Я]+/.exec(src.slice(i))!
      // «xsin» або «2xx» — розбиваємо на x і решту, щоб працювало неявне множення
      let word = m[0].toLowerCase()
      let at = i
      while (word.startsWith('x') && word !== 'x' && !FUNCTIONS.some((f) => word
   === f) && !(word in ALIASES)) {
        tokens.push({ t: 'id', v: 'x', pos: at })
        word = word.slice(1)
        at++
      }
      tokens.push({ t: 'id', v: word, pos: at })
      i += m[0].length
      continue
    }
    const sym = NORMALIZE[c] ?? c
    if (sym.length > 1 && sym.startsWith('^')) {
```

```
      tokens.push({ t: 'sym', v: '^', pos: i }, { t: 'num', v:
    Number(sym.slice(1)), pos: i })
    } else if (sym === 'pi') {
      tokens.push({ t: 'id', v: 'pi', pos: i })
    } else if ('+-*/^(),'.includes(sym)) {
      tokens.push({ t: 'sym', v: sym, pos: i })
    } else {
      throw new FuncError(`Недопустимий символ «${c}» на позиції ${i + 1}`, i)
    }
    i++
  }
  tokens.push({ t: 'end', pos: src.length })
  return tokens
}

export function parseFunction(src: string): Node {
  if (!src.trim()) throw new FuncError('Введіть формулу', 0)
  const tokens = tokenize(src)
  let p = 0
  const peek = () => tokens[p]
  const isSym = (v: string) => peek().t === 'sym' && (peek() as { v: string }).v
    === v
  const expect = (v: string) => {
    if (!isSym(v)) throw new FuncError(`Очікується «${v}» на позиції ${peek().pos
    + 1}`, peek().pos)
    p++
  }

  function expr(): Node {
    let a = term()
    while (isSym('+') || isSym('-')) {
      const op = (tokens[p++] as { v: '+' | '-' }).v
      a = { k: 'bin', op, a, b: term() }
    }
    return a
  }
  // Неявне множення: 2x, 2sin(x), x(x+1), (x+1)(x−1)
  const startsPrimary = () => peek().t === 'num' || peek().t === 'id' ||
    isSym('(')
  function term(): Node {
    let a = unary()
    for (;;) {
      if (isSym('*') || isSym('/')) {
        const op = (tokens[p++] as { v: '*' | '/' }).v
        a = { k: 'bin', op, a, b: unary() }
      } else if (startsPrimary()) {
        a = { k: 'bin', op: '*', a, b: power() }
      } else return a
    }
  }
  function unary(): Node {
    if (isSym('-')) { p++; return { k: 'neg', a: unary() } }
    if (isSym('+')) { p++; return unary() }
    return power()
  }
  function power(): Node {
    const a = primary()
    if (isSym('^')) { p++; return { k: 'pow', a, b: unary() } }
    return a
  }
  function primary(): Node {
    const tok = peek()
    if (tok.t === 'num') { p++; return { k: 'num', v: tok.v } }
```

```
    if (isSym('(')) {
      p++
      const e = expr()
      expect(')')
      return e
    }
    if (tok.t === 'id') {
      p++
      if (tok.v === 'x') return { k: 'x' }
      if (tok.v === 'pi') return { k: 'num', v: Math.PI }
      if (tok.v === 'e') return { k: 'num', v: Math.E }
      const name = (ALIASES[tok.v] ?? tok.v) as FnName
      if (!FUNCTIONS.includes(name)) throw new FuncError(`Невідома назва «${tok.v}» на позиції ${tok.pos + 1}`, tok.pos)
      // sin^2(x)
      let exponent: Node | null = null
      if (isSym('^')) {
        p++
        const e = peek()
        if (e.t !== 'num') throw new FuncError(`Після «${tok.v}^» очікується число`, e.pos)
        p++
        exponent = { k: 'num', v: e.v }
      }
      if (!isSym('(')) throw new FuncError(`Після «${tok.v}» очікується «(» на позиції ${peek().pos + 1}`, peek().pos)
      p++
      const args = [expr()]
      while (isSym(',')) { p++; args.push(expr()) }
      expect(')')
      const need = name === 'root' ? 2 : 1
      if (args.length !== need) {
        throw new FuncError(name === 'root' ? 'root(вираз, n) має два аргументи' : `${name}(…) має один аргумент`, tok.pos)
      }
      if (name === 'root') {
        const n = args[1]
        if (n.k !== 'num' || !Number.isInteger(n.v) || n.v < 2) throw new FuncError('Степінь кореня root(вираз, n) — ціле число від 2', tok.pos)
      }
      const call: Node = { k: 'fn', name, args }
      return exponent ? { k: 'pow', a: call, b: exponent } : call
    }
    if (tok.t === 'end') throw new FuncError('Вираз обірвався: очікується число, x або «(»', tok.pos)
    throw new FuncError(`Неочікуваний символ «${tok.v}» на позиції ${tok.pos + 1}`, tok.pos)
  }

  const node = expr()
  if (peek().t !== 'end') {
    const t = peek()
    throw new FuncError(`Зайвий символ «${t.t === 'end' ? '' : t.v}» на позиції ${t.pos + 1}`, t.pos)
  }
  return node
}

// ---------- Запис формули (для перегляду та звіту) ----------

const PREC = { add: 1, mul: 2, neg: 3, pow: 4, atom: 5 }
function prec(n: Node): number {
```

```
  if (n.k === 'bin') return n.op === '+' || n.op === '-' ? PREC.add : PREC.mul
  if (n.k === 'neg') return PREC.neg
  if (n.k === 'pow') return PREC.pow
  return n.k === 'num' && n.v < 0 ? PREC.neg : PREC.atom
}
const num = (v: number) => (v === Math.PI ? 'π' : v === Math.E ? 'e' :
    String(v).replace('-', '−'))

export function formatFunction(n: Node): string {
  const wrap = (c: Node, min: number) => (prec(c) < min ? `(${formatFunction(c)})`
    : formatFunction(c))
  switch (n.k) {
    case 'num': return num(n.v)
    case 'x': return 'x'
    case 'neg': return `−${wrap(n.a, PREC.neg)}`
    case 'pow': return `${wrap(n.a, PREC.atom)}^${wrap(n.b, PREC.atom)}`
    case 'fn': return `${n.name}(${n.args.map(formatFunction).join(', ')})`
    case 'bin': {
      const p = prec(n)
      const sign = { '+': ' + ', '-': ' − ', '*': '·', '/': '/' }[n.op]
      // права частина віднімання/ділення — у дужках, якщо того самого рівня
      return `${wrap(n.a, p)}${sign}${wrap(n.b, n.op === '-' || n.op === '/' ? p +
    1 : p)}`
    }
  }
}

// ---------- Обчислення в точці ----------

function rootReal(u: number, n: number): number {
  if (u < 0) {
    if (n % 2 === 0) throw new DomainError(`корінь парного степеня ${n} з
    від'ємного числа`)
    return -((-u) ** (1 / n))
  }
  return u ** (1 / n)
}

function powReal(a: number, b: number): number {
  if (Number.isInteger(b)) return a ** b
  if (a < 0) throw new DomainError('нецілий степінь від'ємного числа —
    використайте root(вираз, n)')
  return a ** b
}

export function evalAt(n: Node, x: number): number {
  const r = (m: Node) => evalAt(m, x)
  let v: number
  switch (n.k) {
    case 'num': return n.v
    case 'x': return x
    case 'neg': return -r(n.a)
    case 'pow': v = powReal(r(n.a), r(n.b)); break
    case 'bin': {
      const a = r(n.a)
      const b = r(n.b)
      if (n.op === '/' && b === 0) throw new DomainError('ділення на 0')
      v = n.op === '+' ? a + b : n.op === '-' ? a - b : n.op === '*' ? a * b : a /
    b
      break
    }
    case 'fn': {
```

```
      const u = r(n.args[0])
      switch (n.name) {
        case 'sin': v = Math.sin(u); break
        case 'cos': v = Math.cos(u); break
        case 'tan': v = Math.tan(u); break
        case 'ctg': v = Math.cos(u) / Math.sin(u); break
        case 'exp': v = Math.exp(u); break
        case 'abs': v = Math.abs(u); break
        case 'ln':
        case 'lg':
          if (u <= 0) throw new DomainError(`${n.name} від недодатного числа`)
          v = n.name === 'ln' ? Math.log(u) : Math.log10(u)
          break
        case 'sqrt':
          if (u < 0) throw new DomainError('квадратний корінь з від’ємного числа')
          v = Math.sqrt(u)
          break
        case 'root': v = rootReal(u, (n.args[1] as { v: number }).v); break
      }
    }
  }
  if (!Number.isFinite(v)) throw new DomainError('значення нескінченне')
  return v
}

// ---------- Обчислення на інтервалі ----------

/** Розширення на кілька ulp: Math.sin, Math.exp тощо не мають гарантованого
    округлення */
function widen(lo: number, hi: number): Interval {
  const d = (x: number) => Math.abs(x) * 4e-16 + 1e-300
  if (!Number.isFinite(lo) || !Number.isFinite(hi)) throw new
    DomainError('значення нескінченне')
  return new Interval(lo - d(lo), hi + d(hi))
}

const mono = (X: Interval, f: (t: number) => number): Interval => widen(f(X.lo),
    f(X.hi))

/** Чи є в [lo; hi] точка виду c + k·period */
function hits(X: Interval, c: number, period: number): boolean {
  return Math.ceil((X.lo - c) / period) <= Math.floor((X.hi - c) / period)
}

function sinI(X: Interval): Interval {
  if (X.width >= 2 * Math.PI) return new Interval(-1, 1)
  const a = Math.sin(X.lo)
  const b = Math.sin(X.hi)
  const lo = hits(X, -Math.PI / 2, 2 * Math.PI) ? -1 : Math.min(a, b)
  const hi = hits(X, Math.PI / 2, 2 * Math.PI) ? 1 : Math.max(a, b)
  const w = widen(lo, hi)
  return new Interval(Math.max(-1, w.lo), Math.min(1, w.hi))
}

const cosI = (X: Interval): Interval => sinI(X.add(Math.PI / 2))

function powIntI(X: Interval, n: number): Interval {
  if (n === 0) return new Interval(1)
  if (n < 0) return divI(new Interval(1), powIntI(X, -n))
  if (n % 2 === 1) return mono(X, (t) => t ** n)
  const a = Math.abs(X.lo) ** n
```

```
  const b = Math.abs(X.hi) ** n
  return X.contains(0) ? new Interval(0, widen(0, Math.max(a, b)).hi) :
    widen(Math.min(a, b), Math.max(a, b))
}

function divI(A: Interval, B: Interval): Interval {
  if (B.contains(0)) throw new DomainError(`ділення на інтервал ${B}, що містить
    0`)
  const q = A.quotients(B)
  return widen(Math.min(...q), Math.max(...q))
}

function lnI(X: Interval): Interval {
  if (X.lo <= 0) throw new DomainError(`ln на інтервалі ${X}, що містить недодатні
    числа`)
  return mono(X, Math.log)
}

function powI(A: Interval, B: Interval, bNode: Node): Interval {
  // цілий сталий показник: x^3, (x−2)^3, sin(x)^2
  if (bNode.k === 'num' && Number.isInteger(bNode.v)) return powIntI(A, bNode.v)
  if (A.lo <= 0) throw new DomainError(`нецілий степінь інтервалу ${A}, що містить
    недодатні числа`)
  // A^B = exp(B·ln A)
  return mono(B.mul(lnI(A)), Math.exp)
}

export function evalInterval(n: Node, X: Interval): Interval {
  const r = (m: Node) => evalInterval(m, X)
  switch (n.k) {
    case 'num': return new Interval(n.v)
    case 'x': return X
    case 'neg': return r(n.a).neg()
    case 'pow': return powI(r(n.a), r(n.b), n.b)
    case 'bin': {
      const a = r(n.a)
      const b = r(n.b)
      if (n.op === '/') return divI(a, b)
      const v = n.op === '+' ? a.add(b) : n.op === '-' ? a.sub(b) : a.mul(b)
      return widen(v.lo, v.hi)
    }
    case 'fn': {
      const U = r(n.args[0])
      switch (n.name) {
        case 'sin': return sinI(U)
        case 'cos': return cosI(U)
        case 'tan':
          if (hits(U, Math.PI / 2, Math.PI)) throw new DomainError(`tan на
    інтервалі ${U}: функція має розрив`)
          return mono(U, Math.tan)
        case 'ctg':
          if (hits(U, 0, Math.PI)) throw new DomainError(`ctg на інтервалі ${U}:
    функція має розрив`)
          // на одній гілці ctg спадає
          return widen(1 / Math.tan(U.hi), 1 / Math.tan(U.lo))
        case 'exp': return mono(U, Math.exp)
        case 'abs':
          return U.contains(0) ? widen(0, Math.max(-U.lo, U.hi)) :
    widen(Math.min(Math.abs(U.lo), Math.abs(U.hi)), Math.max(Math.abs(U.lo),
    Math.abs(U.hi)))
        case 'ln': return lnI(U)
        case 'lg': return mono(lnI(U), (t) => t / Math.LN10)
```

```
        case 'sqrt':
          if (U.lo < 0) throw new DomainError(`квадратний корінь на інтервалі ${U}, що містить від’ємні числа`)
          return mono(U, Math.sqrt)
        case 'root': {
          const k = (n.args[1] as { v: number }).v
          if (k % 2 === 0 && U.lo < 0) throw new DomainError(`корінь парного степеня на інтервалі ${U}, що містить від’ємні числа`)
          return mono(U, (t) => rootReal(t, k))
        }
      }
    }
  }
}

/** Похідна, введена користувачем, порівнюється з чисельною (центральна різниця) */
export function derivativeMismatch(f: Node, df: Node, a: number, b: number, points = 21): number | null {
  for (let i = 0; i < points; i++) {
    const x = a + ((b - a) * (i + 0.5)) / points
    const h = 1e-6 * Math.max(1, Math.abs(x))
    try {
      const numeric = (evalAt(f, x + h) - evalAt(f, x - h)) / (2 * h)
      const given = evalAt(df, x)
      if (Math.abs(numeric - given) > 1e-4 * Math.max(1, Math.abs(numeric)))
    return x
    } catch {
      // точку поза областю визначення пропускаємо — про неї повідомить перевірка області
    }
  }
  return null
}
```